

Fitness Company Database System

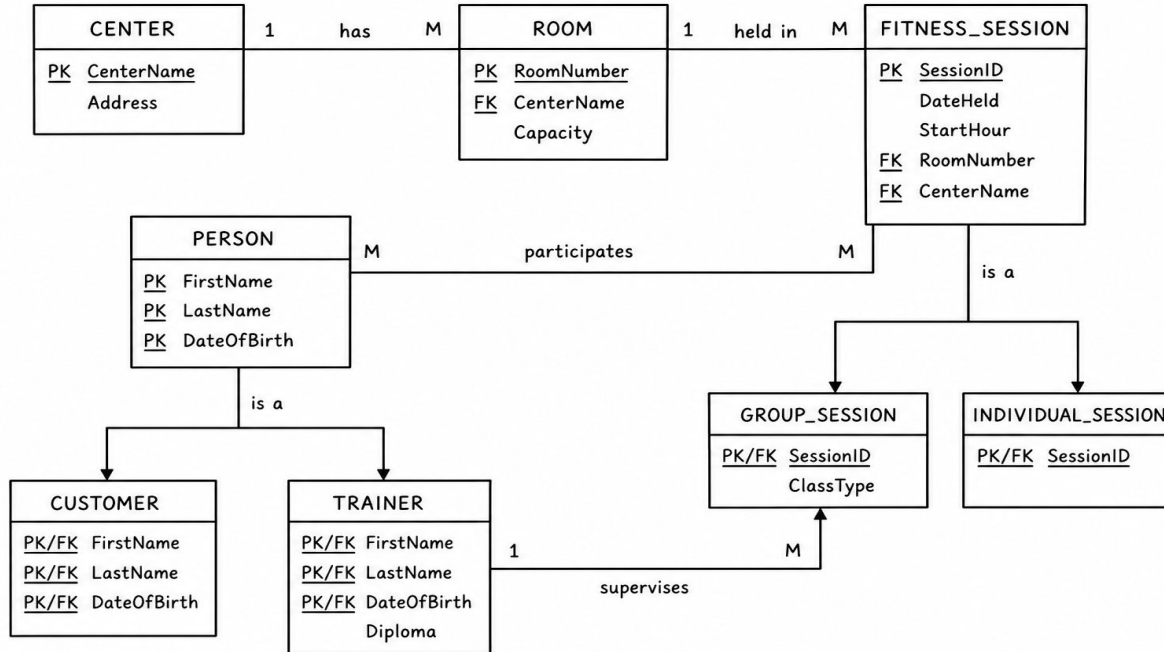
Justin Garrett

CSCI 0340

Project Goal

This project focuses on designing and implementing a database system for a fitness company. The goal is to organize and manage data related to centers, rooms, members, trainers, and sessions.

ER Diagram



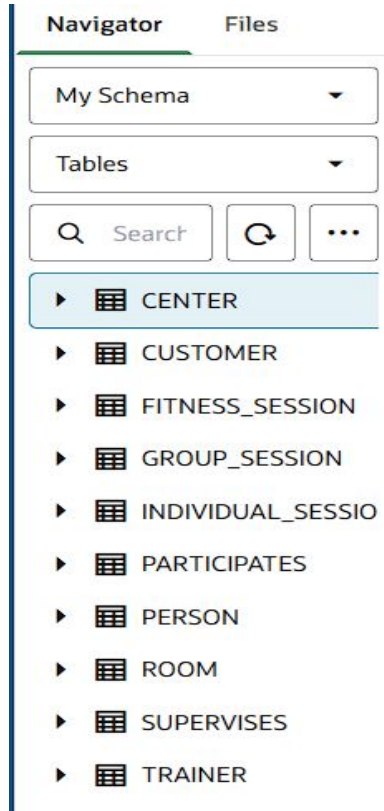
My ER model represents the system at a conceptual level. The main entities are Center, Room, Person, and Fitness Session.

The relationships show how these entities connect. For example, a center has many rooms, and a room can host many sessions.

A person can participate in multiple sessions, creating a many-to-many relationship.

The model also includes specialization, where a person can be either a customer or a trainer, and sessions can be group or individual.

Relational Schema & Implementation



I converted the ER model into a relational schema by creating tables such as CENTER, ROOM, PERSON, TRAINER, CUSTOMER, and FITNESS_SESSION.

I also created supporting tables like GROUP_SESSION, INDIVIDUAL_SESSION, PARTICIPATES, and SUPERVISES to represent relationships.

These tables were implemented in Oracle using primary and foreign keys to maintain data integrity.

[SQL Worksheet]*



Aa



```
1 CREATE TABLE GROUP_SESSION (  
2     SessionID NUMBER PRIMARY KEY,  
3     ClassType VARCHAR2(50),  
4     FOREIGN KEY (SessionID) REFERENCES SESSION(SessionID)  
5 );  
6  
7 CREATE TABLE INDIVIDUAL_SESSION (  
8     SessionID NUMBER PRIMARY KEY,  
9     FOREIGN KEY (SessionID) REFERENCES SESSION(SessionID)  
10 );  
11  
12 CREATE TABLE PARTICIPATES (  
13     FirstName VARCHAR2(30),  
14     LastName VARCHAR2(30),  
15     DateOfBirth DATE,  
16     SessionID NUMBER,  
17     PRIMARY KEY (FirstName, LastName, DateOfBirth, SessionID),  
18     FOREIGN KEY (SessionID) REFERENCES SESSION(SessionID)  
19 );  
20  
21 CREATE TABLE SUPERVISES (  
22     FirstName VARCHAR2(30),  
23     LastName VARCHAR2(30),  
24     DateOfBirth DATE,  
25     SessionID NUMBER UNIQUE,  
26     FOREIGN KEY (SessionID) REFERENCES GROUP_SESSION(SessionID)  
27 );
```

SQL Worksheet

```
1 CREATE TABLE CENTER (  
2     CenterName VARCHAR2(50) PRIMARY KEY,  
3     Address VARCHAR2(100)  
4 );  
5  
6 CREATE TABLE ROOM (  
7     RoomNumber NUMBER,  
8     CenterName VARCHAR2(50),  
9     Capacity NUMBER,  
10    PRIMARY KEY (RoomNumber, CenterName),  
11    FOREIGN KEY (CenterName) REFERENCES CENTER(CenterName)  
12 );  
13  
14 CREATE TABLE PERSON (  
15     FirstName VARCHAR2(30),  
16     LastName VARCHAR2(30),  
17     DateOfBirth DATE,  
18     PRIMARY KEY (FirstName, LastName, DateOfBirth)  
19 );  
20  
21 CREATE TABLE TRAINER (  
22     FirstName VARCHAR2(30),  
23     LastName VARCHAR2(30),  
24     DateOfBirth DATE,  
25     Diploma VARCHAR2(50),  
26     PRIMARY KEY (FirstName, LastName, DateOfBirth),  
27     FOREIGN KEY (FirstName, LastName, DateOfBirth)  
28     REFERENCES PERSON(FirstName, LastName, DateOfBirth)  
29 );  
30  
31 CREATE TABLE CUSTOMER (  
32     FirstName VARCHAR2(30),  
33     LastName VARCHAR2(30),  
34     DateOfBirth DATE,  
35     PRIMARY KEY (FirstName, LastName, DateOfBirth),  
36     FOREIGN KEY (FirstName, LastName, DateOfBirth)  
37     REFERENCES PERSON(FirstName, LastName, DateOfBirth)  
38 );  
39
```

SQL Worksheet

Insert Data

```
[SQL Worksheet]*  ▶ ⌵ 🔍 📄 ⌵ Aa ▼ 🗑️ ⬆️
1  INSERT INTO CENTER VALUES ('FitPlaza', '123 Main Street');
2  INSERT INTO CENTER VALUES ('My6Pack', '45 Health Avenue');
3  INSERT INTO CENTER VALUES ('PowerHouse', '88 Gym Road');
4
5  INSERT INTO ROOM VALUES (1, 'FitPlaza', 30);
6  INSERT INTO ROOM VALUES (2, 'FitPlaza', 20);
7  INSERT INTO ROOM VALUES (1, 'My6Pack', 25);
8  INSERT INTO ROOM VALUES (2, 'My6Pack', 15);
9  INSERT INTO ROOM VALUES (1, 'PowerHouse', 40);
10
11 INSERT INTO PERSON VALUES ('John', 'Smith', DATE '1998-04-12');
12 INSERT INTO PERSON VALUES ('Mary', 'Jones', DATE '1995-07-20');
13 INSERT INTO PERSON VALUES ('David', 'Brown', DATE '1990-02-15');
14 INSERT INTO PERSON VALUES ('Lisa', 'Green', DATE '1999-11-03');
15 INSERT INTO PERSON VALUES ('Chris', 'White', DATE '1988-09-25');
16 INSERT INTO PERSON VALUES ('Angela', 'Davis', DATE '1992-06-10');
17 INSERT INTO PERSON VALUES ('Marcus', 'Hill', DATE '1996-01-18');
18 INSERT INTO PERSON VALUES ('Tina', 'Moore', DATE '1994-12-05');
19
20 INSERT INTO CUSTOMER VALUES ('John', 'Smith', DATE '1998-04-12');
21 INSERT INTO CUSTOMER VALUES ('Mary', 'Jones', DATE '1995-07-20');
22 INSERT INTO CUSTOMER VALUES ('Lisa', 'Green', DATE '1999-11-03');
23 INSERT INTO CUSTOMER VALUES ('Marcus', 'Hill', DATE '1996-01-18');
24 INSERT INTO CUSTOMER VALUES ('Tina', 'Moore', DATE '1994-12-05');
25
26 INSERT INTO TRAINER VALUES ('David', 'Brown', DATE '1990-02-15',
27 INSERT INTO TRAINER VALUES ('Chris', 'White', DATE '1988-09-25',
28 INSERT INTO TRAINER VALUES ('Angela', 'Davis', DATE '1992-06-10',
29
30 INSERT INTO FITNESS_SESSION VALUES (101, DATE '2026-05-01', 9, 1);
31 INSERT INTO FITNESS_SESSION VALUES (102, DATE '2026-05-01', 10,
32 INSERT INTO FITNESS_SESSION VALUES (103, DATE '2026-05-02', 11,
33 INSERT INTO FITNESS_SESSION VALUES (104, DATE '2026-05-02', 12,
34 INSERT INTO FITNESS_SESSION VALUES (105, DATE '2026-05-03', 8, 1);
35
36 INSERT INTO GROUP_SESSION VALUES (101, 'Aerobics');
37 INSERT INTO GROUP_SESSION VALUES (102, 'Body Styling');
38 INSERT INTO GROUP_SESSION VALUES (103, 'Strength Training');
```

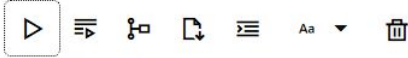
After creating the tables, I inserted data into all tables including CENTER, ROOM, PERSON, CUSTOMER, TRAINER, and FITNESS_SESSION.

I also linked people to sessions using PARTICIPATES and assigned trainers using SUPERVISES.

This step is important because without data, queries would return empty results

SQL Queries

[SQL Worksheet]*



```
1 SELECT * FROM FITNESS_SESSION;
```

Query result

Script output

DBMS output

Explain Plan

SQL history



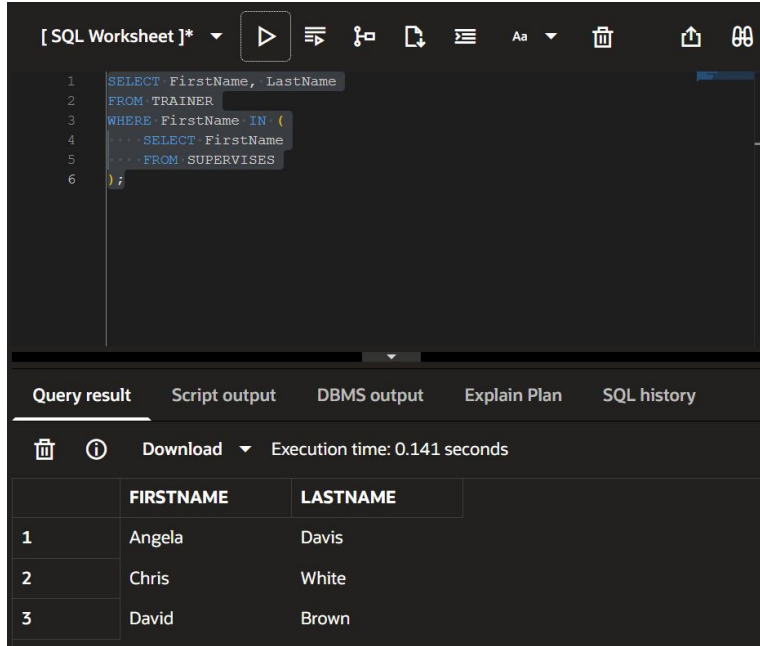
Download

Execution time: 0.001 seconds

	SESSIONID	DATEHELD	STARTHOUR	ROOMNUMBER	CENTERNAME
1	101	5/1/2026, 12:00:00	9	1	FitPlaza
2	102	5/1/2026, 12:00:00	10	2	FitPlaza
3	103	5/2/2026, 12:00:00	11	1	My6Pack
4	104	5/2/2026, 12:00:00	12	2	My6Pack
5	105	5/3/2026, 12:00:00	8	1	PowerHouse

After inserting data, I used SQL queries such as SELECT, JOIN, and aggregation to retrieve and analyze information. I also implemented advanced queries including nested queries, correlated queries, NOT EXISTS, and set operations. For example, a SELECT query retrieves all session data from the database.

Nested query



The screenshot shows an SQL IDE interface. The top toolbar includes icons for running the query, saving, and other standard functions. The main editor area contains the following SQL code:

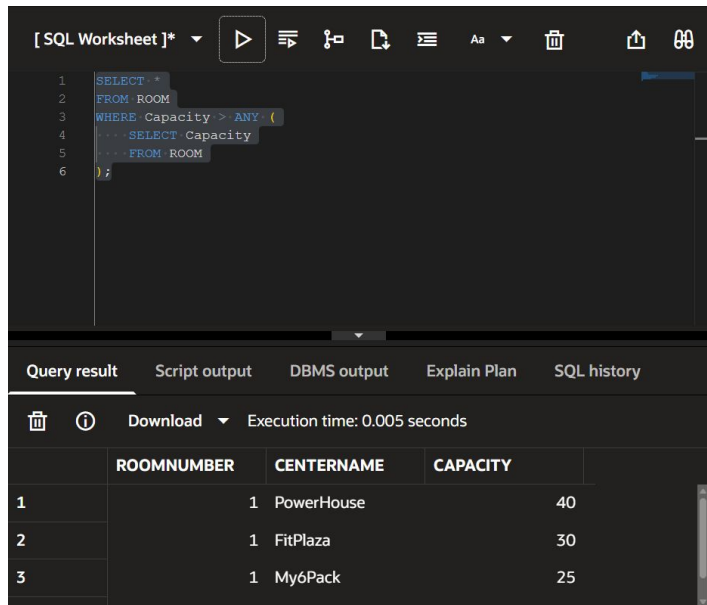
```
1 SELECT FirstName, LastName
2 FROM TRAINER
3 WHERE FirstName IN (
4     SELECT FirstName
5     FROM SUPERVISES
6 );
```

Below the editor, there are tabs for 'Query result', 'Script output', 'DBMS output', 'Explain Plan', and 'SQL history'. The 'Query result' tab is active, displaying the results of the query. It shows a table with two columns: 'FIRSTNAME' and 'LASTNAME'. The results are as follows:

	FIRSTNAME	LASTNAME
1	Angela	Davis
2	Chris	White
3	David	Brown

This query uses a nested query to find trainers who supervise sessions. The inner query selects names from the SUPERVISES table, and the outer query returns matching trainers

Any query



The screenshot shows a SQL worksheet interface. The top toolbar includes a play button, a list icon, a refresh icon, a save icon, a text editor icon, a font size dropdown, a trash icon, an upload icon, and a help icon. The SQL editor contains the following query:

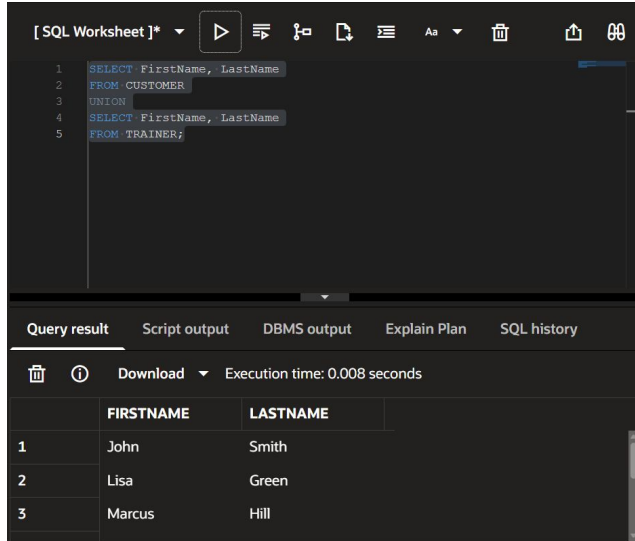
```
1 SELECT *
2 FROM ROOM
3 WHERE Capacity > ANY (
4     SELECT Capacity
5     FROM ROOM
6 )
```

Below the editor, the 'Query result' tab is active, showing the execution time as 0.005 seconds. The results are displayed in a table with three columns: ROOMNUMBER, CENTERNAME, and CAPACITY.

	ROOMNUMBER	CENTERNAME	CAPACITY
1	1	PowerHouse	40
2	1	FitPlaza	30
3	1	MyóPack	25

This query returns rooms with a capacity greater than at least one other room. The ANY keyword compares values against a set of results.

UNION (Set Operation)



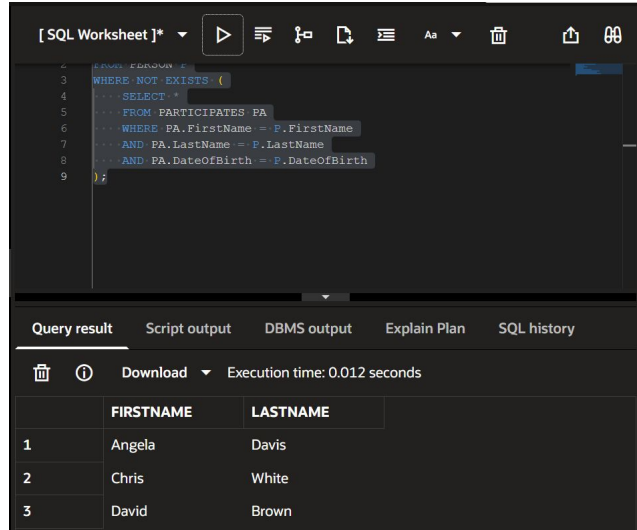
The screenshot shows an SQL IDE interface. The top panel displays a SQL query using the UNION operator to combine results from the CUSTOMER and TRAINER tables. The bottom panel shows the query results in a table format, with columns FIRSTNAME and LASTNAME. The results are: John Smith, Lisa Green, and Marcus Hill. The execution time is 0.008 seconds.

```
1 SELECT FirstName, LastName
2 FROM CUSTOMER
3 UNION
4 SELECT FirstName, LastName
5 FROM TRAINER;
```

	FIRSTNAME	LASTNAME
1	John	Smith
2	Lisa	Green
3	Marcus	Hill

This query combines the results of two queries into one result set. It returns all unique names from both the CUSTOMER and TRAINER tables, removing duplicates.

Correlated Query (EXISTS)



```
2  FROM PERSON P
3  WHERE NOT EXISTS (
4  ... SELECT *
5  ... FROM PARTICIPATES PA
6  ... WHERE PA.FirstName = P.FirstName
7  ... AND PA.LastName = P.LastName
8  ... AND PA.DateOfBirth = P.DateOfBirth
9  );
```

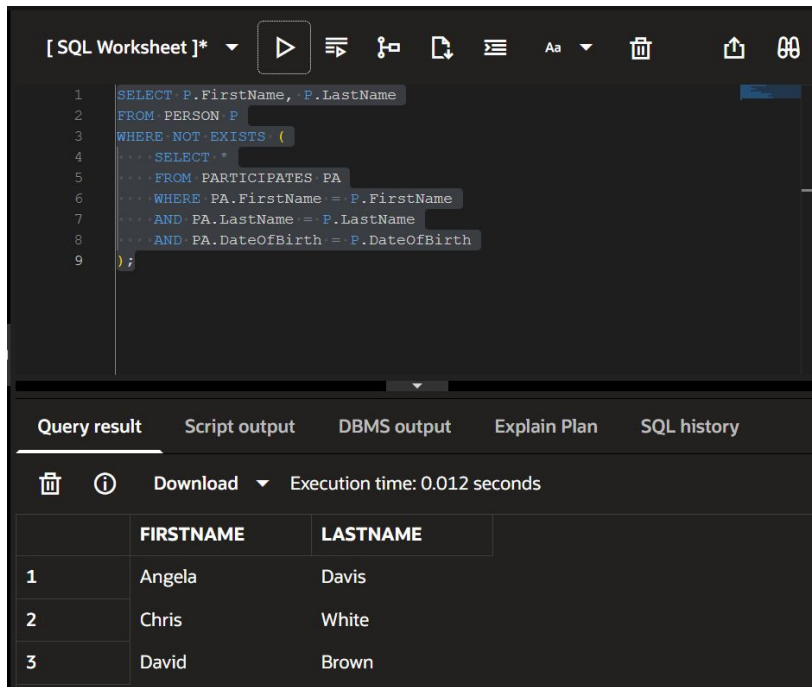
Query result Script output DBMS output Explain Plan SQL history

Download Execution time: 0.012 seconds

	FIRSTNAME	LASTNAME
1	Angela	Davis
2	Chris	White
3	David	Brown

This correlated query checks for each trainer if they exist in the SUPERVISES table. It returns only trainers who are currently supervising sessions.

NOT EXISTS Query



The screenshot shows an SQL IDE with a query editor and a results pane. The query in the editor is:

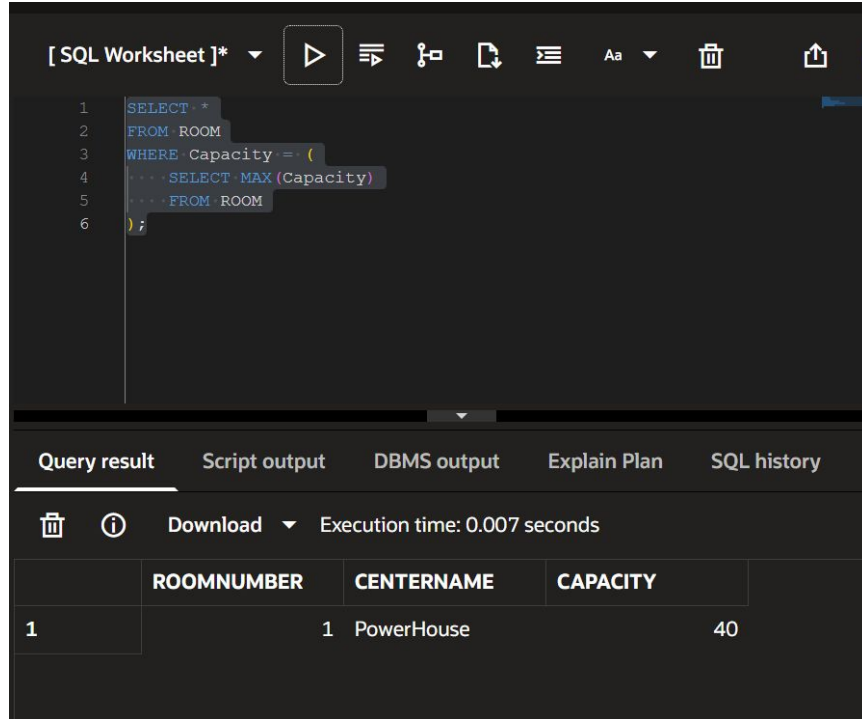
```
1 SELECT P.FirstName, P.LastName
2 FROM PERSON P
3 WHERE NOT EXISTS (
4     SELECT *
5     FROM PARTICIPATES PA
6     WHERE PA.FirstName = P.FirstName
7     AND PA.LastName = P.LastName
8     AND PA.DateOfBirth = P.DateOfBirth
9 );
```

The results pane shows the following table:

	FIRSTNAME	LASTNAME
1	Angela	Davis
2	Chris	White
3	David	Brown

This query finds people who have not participated in any sessions. It uses NOT EXISTS to check for the absence of matching records in the PARTICIPATES table.

Nested Query (MAX)



The screenshot shows a SQL worksheet interface with a dark theme. The top toolbar includes a dropdown menu labeled "[SQL Worksheet]*", a play button, and several icons for query execution and management. The SQL editor contains the following code:

```
1 SELECT *
2 FROM ROOM
3 WHERE Capacity = (
4     SELECT MAX(Capacity)
5     FROM ROOM
6 ) ;
```

Below the editor, there are tabs for "Query result", "Script output", "DBMS output", "Explain Plan", and "SQL history". The "Query result" tab is active, showing a table with the following data:

	ROOMNUMBER	CENTERNAME	CAPACITY
1	1	PowerHouse	40

This query finds the room with the highest capacity by using a nested query with the MAX function.

JOIN Query

[SQL Worksheet]*       Aa 

```
1 SELECT F.SessionID, F.CenterName, R.Capacity
2 FROM FITNESS_SESSION F
3 JOIN ROOM R
4 ON F.RoomNumber = R.RoomNumber
5 AND F.CenterName = R.CenterName;
```

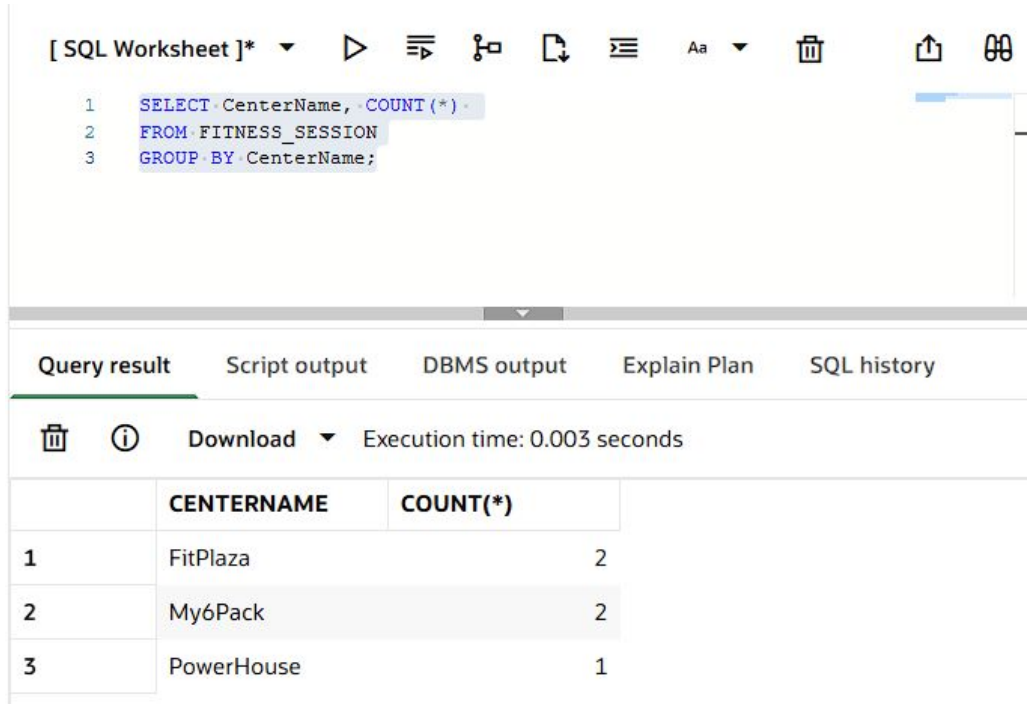
Query result Script output DBMS output Explain Plan SQL hi

  Download  Execution time: 0.011 seconds

	SESSIONID	CENTERNAME	CAPACITY
1	101	FitPlaza	30
2	102	FitPlaza	20
3	103	My6Pack	25
4	104	My6Pack	15
5	105	PowerHouse	40

This JOIN query combines data from the FITNESS_SESSION and ROOM tables. It uses matching room number and center name to connect the tables. The result shows each session along with the room capacity.

Aggregation Query



The screenshot shows a SQL worksheet interface. At the top, there's a toolbar with various icons for execution, formatting, and saving. Below the toolbar, the SQL query is entered in a text area:

```
1 SELECT CenterName, COUNT(*)
2 FROM FITNESS_SESSION
3 GROUP BY CenterName;
```

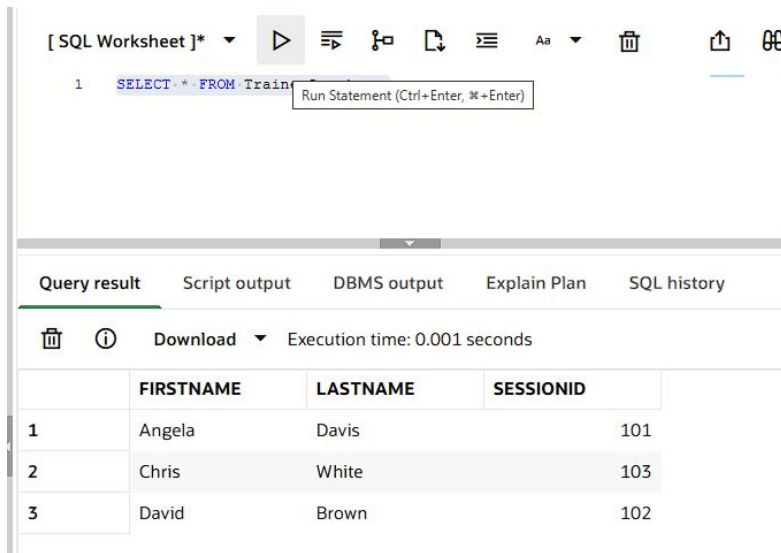
Below the query editor, there are tabs for 'Query result', 'Script output', 'DBMS output', 'Explain Plan', and 'SQL history'. The 'Query result' tab is selected, showing the execution time as 0.003 seconds. Below this, a table displays the results of the query:

	CENTERNAME	COUNT(*)
1	FitPlaza	2
2	My6Pack	2
3	PowerHouse	1

This aggregation query uses COUNT and GROUP BY to summarize the data. It shows how many sessions are held at each center. For example, FitPlaza and My6Pack each have two sessions, while PowerHouse has one.

Views (Virtual Tables)

Views simplify complex queries by storing them as reusable virtual tables. I created views such as TrainerSessions and SessionParticipants to make it easier to access information about trainers, sessions, and participants. I also created a CenterSessions view to summarize the total number of sessions at each center.

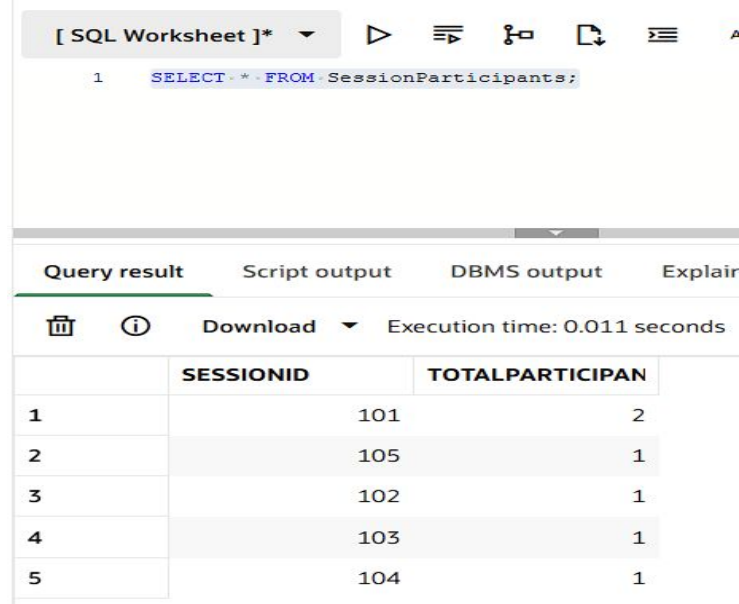


The screenshot shows a SQL Worksheet interface. The query editor at the top contains the following SQL statement:

```
1 SELECT * FROM TrainerSessions
```

A tooltip "Run Statement (Ctrl+Enter, ⌘+Enter)" is visible over the "Run" button (a play icon). Below the editor, the "Query result" tab is selected, displaying a table with the following data:

	FIRSTNAME	LASTNAME	SESSIONID
1	Angela	Davis	101
2	Chris	White	103
3	David	Brown	102



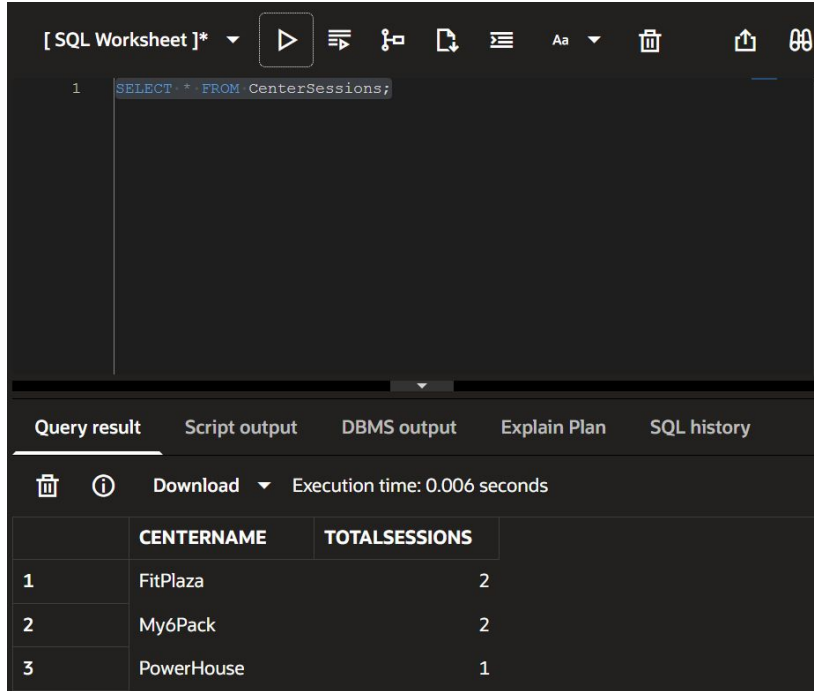
The screenshot shows a SQL Worksheet interface. The query editor at the top contains the following SQL statement:

```
1 SELECT * FROM SessionParticipants
```

Below the editor, the "Query result" tab is selected, displaying a table with the following data:

	SESSIONID	TOTALPARTICIPAN
1	101	2
2	105	1
3	102	1
4	103	1
5	104	1

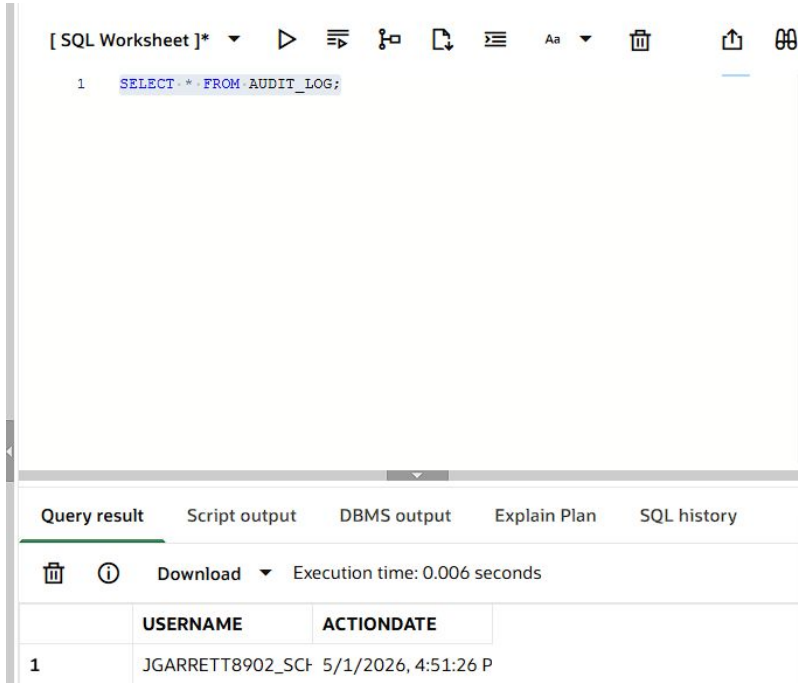
Views



The screenshot shows an SQL IDE interface. At the top, there's a toolbar with icons for running queries, saving, and other functions. Below the toolbar, the SQL editor contains the query: `SELECT * FROM CenterSessions;`. The query is executed, and the results are displayed in a table below. The table has two columns: **CENTERNAME** and **TOTALSESSIONS**. The results show three rows of data.

	CENTERNAME	TOTALSESSIONS
1	FitPlaza	2
2	My6Pack	2
3	PowerHouse	1

Trigger (Audit Logging) challenge



The screenshot shows a SQL worksheet interface. At the top, there is a toolbar with various icons for execution and editing. Below the toolbar, a SQL query is entered: `SELECT * FROM AUDIT_LOG;`. The query is executed, and the results are displayed in a table below. The table has two columns: **USERNAME** and **ACTIONDATE**. The first row of data shows the username `JGARRETT8902_SCF` and the action date `5/1/2026, 4:51:26 P`. The execution time is noted as 0.006 seconds.

	USERNAME	ACTIONDATE
1	JGARRETT8902_SCF	5/1/2026, 4:51:26 P

This screenshot shows the result of my trigger. After updating the customer table, the trigger automatically inserted a record into the audit log table. It stores the username and the date and time of the update.

Conclusion

In conclusion, this project demonstrates how databases can be used to efficiently manage real-world systems. I designed and implemented a complete database, applied ER modeling and relational schema, and used SQL to store, retrieve, and analyze data.

Overall, this project strengthened my understanding of database design and its real-world applications.